

TP 8 — Points, vecteurs et couleurs

Semaine 8 ■ Fichiers dans NSI/TP08/ ■ 2 h

Objectifs

- Manipuler des tuples dans de vraies fonctions
- Déconstruire systématiquement plutôt qu'indexer
- Composer des fonctions qui échangent des tuples

Partie A — Géométrie — `geometrie.py`

(40 min)

Un point est un couple (x, y) . Écrire et tester :

1. `translation(p, v)` — le point `p` déplacé du vecteur `v = (dx, dy)`.
2. `distance(a, b)` — réécrire la fonction du TP 3, mais avec **deux couples** en paramètres (déconstruire!).
3. `perimetre_triangle(a, b, c)` — en **appelant distance** trois fois.
4. `symetrique(p)` — le symétrique de `p` par rapport à l'axe des abscisses.
5. ★ `plus_proche(p, a, b)` — renvoie celui des points `a` ou `b` qui est le plus proche de `p`.

Tests conseillés : `translation((1, 2), (3, -1)) → (4, 1)` ; `distance((0, 0), (3, 4)) → 5.0`.

Partie B — Couleurs RVB — `couleurs.py`

(40 min)

Une couleur est un triplet d'entiers de 0 à 255, comme `(255, 102, 0)`.

1. `est_valide(c)` — `True` si les trois composantes sont dans `[0, 255]`.
2. `negatif(c)` — la couleur inversée : chaque composante `v` devient `255 - v`.
3. `gris(c)` — le gris de même luminosité : les trois composantes prennent la valeur moyenne (entière) des composantes de `c`.
4. `melange(c1, c2)` — la moyenne composante par composante.
5. ★ `vers_hexa(c)` — le code Web de la couleur : `(255, 102, 0)` donne `"#FF6600"` (réutiliser `decimal_vers_hexa` du TP 5, en complétant à 2 chiffres).

Partie C — La monnaie rendue — `monnaie.py`

(30 min)

1. `euros_centimes(somme)` — décompose un prix flottant en `(euros, centimes)` : `euros_centimes(7.35) → (7, 35)` (attention aux arrondis flottants : passer par `round(somme * 100)`).
2. `rendu(prix, paye)` — renvoie le triplet (billets de 10, pièces de 2, pièces de 1) qui rend la monnaie `paye - prix` (entiers, méthode gloutonne : d'abord les 10, puis les 2, puis les 1).
3. Vérifier : `rendu(23, 50) → (2, 3, 1)` ($27 = 2 \times 10 + 3 \times 2 + 1$).

Partie D — Défi ★★

(10 min)

`collision(c1, c2)` — deux cercles donnés par `(x, y, rayon)` ; renvoie `True` s'ils se touchent ou se chevauchent (comparer la distance des centres à la somme des rayons — réutiliser `distance`!).